Київський національний університет імені Тараса Шевченка

Кафедра мережевих і Інтернет технологій

Лабораторна робота №1

з дисципліни

«Технології створення програмних продуктів»

Вступ у шаблони проектування. Шаблон Builder та шаблон Factory Method

Група: МІТ-м11

Студентка: Булгакова Анна

Київ 2026

Завдання 1. Створити програму, використовуючи фабричний метод, яка зможе працювати із декількома типами файлів (текстові, графічні), створюючи бажаний тип файлу за вибором користувача у процесі роботи програми.

```
FactoryMethodFiles
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace FactoryMethodFiles
{
    6 references
    abstract class File
    {
        3 references
        public abstract void Open();
    }
}
```

```
FileCreator.cs
FactoryMethodFiles
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace FactoryMethodFiles
{
    3 references
    abstract class FileCreator
    {
        3 references
        public abstract File FactoryMethod();
    }
}
```

```
FileCreator.cs
FactoryMethodFiles
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace FactoryMethodFiles
{
    1 reference
    class ImageFile : File
    {
        2 references
        public override void Open()
        {
            Console.WriteLine("Відкрито графічний файл (.png/.jpg)");
        }
    }
}
```

FileCreator.cs

FactoryMethodFiles | FactoryMethodFiles.I

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace FactoryMethodFiles
{
    1 reference
    class ImageFileCreator : FileCreator
    {
        2 references
        public override File FactoryMethod()
        {
            return new ImageFile();
        }
    }
}
```

TextFile.cs | FileCreator.cs

FactoryMethodFiles | FactoryMethodFiles.TextFile

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace FactoryMethodFiles
{
    1 reference
    class TextFile : File
    {
        2 references
        public override void Open()
        {
            Console.WriteLine("Відкрито текстовий файл (.txt)");
        }
    }
}
```

TextFileCreator.cs | TextFile.cs | FileCreator.cs

FactoryMethodFiles | FactoryMethodFiles.TextFileCrea

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace FactoryMethodFiles
{

    class TextFileCreator : FileCreator
    {
        2 references
        public override File FactoryMethod()
        {
            return new TextFile();
        }
    }
}
```

Program.cs

FactoryMethodFiles

```
using System;

namespace FactoryMethodFiles
{
    0 references
    class Program
    {
        0 references
        static void Main(string[] args)
        {
            Console.WriteLine("Оберіть тип файлу:");
            Console.WriteLine("1 - Текстовий файл");
            Console.WriteLine("2 - Графічний файл");

            string choice = Console.ReadLine();

            FileCreator creator = null;

            if (choice == "1")
                creator = new TextFileCreator();
            else if (choice == "2")
                creator = new ImageFileCreator();
            else
            {
                Console.WriteLine("Невірний вибір");
                return;
            }

            // Factory Method
            File file = creator.FactoryMethod();

            file.Open();

            Console.ReadKey();
        }
    }
}
```

C:\Users\annbu\Source\Repo:

```
Обер?ть тип файлу:
1 - Текстовий файл
2 - Граф?чний файл
1
В?дкрито текстовий файл (.txt)
```

C:\Users\annbu\Source\Repo:

```
Обер?ть тип файлу:
1 - Текстовий файл
2 - Граф?чний файл
2
В?дкрито граф?чний файл (.png/.jpg)
```

Завдання 2. Створити програму MailMessageBuilder, яка буде використовувати шаблон Builder для створення екземпляра електронного листа з можливістю ланцюжкового виклику методів для задання отримувача, відправника, заголовка, вмісту повідомлення.

Program.cs

MailMessageBuilder | MailMessageBuilder.Builder

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace MailMessageBuilder
{
    11 references
    abstract class Builder
    {
        2 references
        public abstract Builder SetFrom(string from);
        2 references
        public abstract Builder SetTo(string to);
        2 references
        public abstract Builder SetSubject(string subject);
        2 references
        public abstract Builder SetBody(string body);
        2 references
        public abstract MailMessage GetResult();
    }
}
```

Program.cs

MailMessageBuilder | MailMessageBuilder.Director

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace MailMessageBuilder
{
    2 references
    class Director
    {
        1 reference
        public MailMessage Construct(
            Builder builder,
            string from,
            string to,
            string subject,
            string body)
        {
            return builder
                .SetFrom(from)
                .SetTo(to)
                .SetSubject(subject)
                .SetBody(body)
                .GetResult();
        }
    }
}
```

Program.cs

MailMessageBuilder | MailMessageBuilder.MailMessage

```
using System;

namespace MailMessageBuilder
{
    class MailMessage
    {
        public string From { get; set; }
        public string To { get; set; }
        public string Subject { get; set; }
        public string Body { get; set; }

        public void Show()
        {
            Console.WriteLine("Від: " + From);
            Console.WriteLine("Кому: " + To);
            Console.WriteLine("Заголовок: " + Subject);
            Console.WriteLine("Повідомлення: " + Body);
        }
    }
}
```

Program.cs

MailMessageBuilder | MailMessageBuilder.MailMessageConcreteBuilder

```
class MailMessageConcreteBuilder : Builder
{
    private MailMessage message = new MailMessage();

    public override Builder SetFrom(string from)
    {
        message.From = from;
        return this;
    }

    public override Builder SetTo(string to)
    {
        message.To = to;
        return this;
    }

    public override Builder SetSubject(string subject)
    {
        message.Subject = subject;
        return this;
    }

    public override Builder SetBody(string body)
    {
        message.Body = body;
        return this;
    }

    public override MailMessage GetResult()
    {
        return message;
    }
}
```

80 % No issues found

```
Program.cs
MailMessageBuilder                                  Program
using MailMessageBuilder;

0 references
class Program
{
    0 references
    static void Main(string[] args)
    {
        Console.WriteLine("Введіть відправника:");
        string from = Console.ReadLine();

        Console.WriteLine("Введіть отримувача:");
        string to = Console.ReadLine();

        Console.WriteLine("Введіть заголовок:");
        string subject = Console.ReadLine();

        Console.WriteLine("Введіть текст повідомлення:");
        string body = Console.ReadLine();

        Builder builder = new MailMessageConcreteBuilder();
        Director director = new Director();

        MailMessage message = director.Construct(
            builder,
            from,
            to,
            subject,
            body);

        Console.WriteLine("\nСтворений лист:");
        message.Show();

        Console.ReadKey();
    }
}
```

Висновки: На даній лабораторній роботі було розглянуто основи шаблонів проєктування та їх роль у розробці програмного забезпечення, а також було ознайомлено із шаблонами, що породжують об’єкти, **Factory Method** та **Builder**, крім того як використовувати їх на практиці. У процесі виконання практичних завдань було реалізовано створення об’єктів різних типів файлів за допомогою фабричного методу, що дозволило відокремити процес створення об’єктів від їх використання. Також було реалізовано побудову складного об’єкта (електронного листа) за допомогою шаблону Builder, що дало змогу гнучко керувати процесом його створення.